

Dynamic Programming

Algorithm Design





西南财经大学
SOUTHWESTERN UNIVERSITY OF FINANCE AND ECONOMICS

Algorithm Design & Analysis

第九讲

西南财经大学

经世济民，孜孜以求



动态规划算法的重要应用

图像压缩

最大子
段和

最优二叉检索
树算法

最优二叉检索
树概念

RNA
二级结
构预测

序列
比对





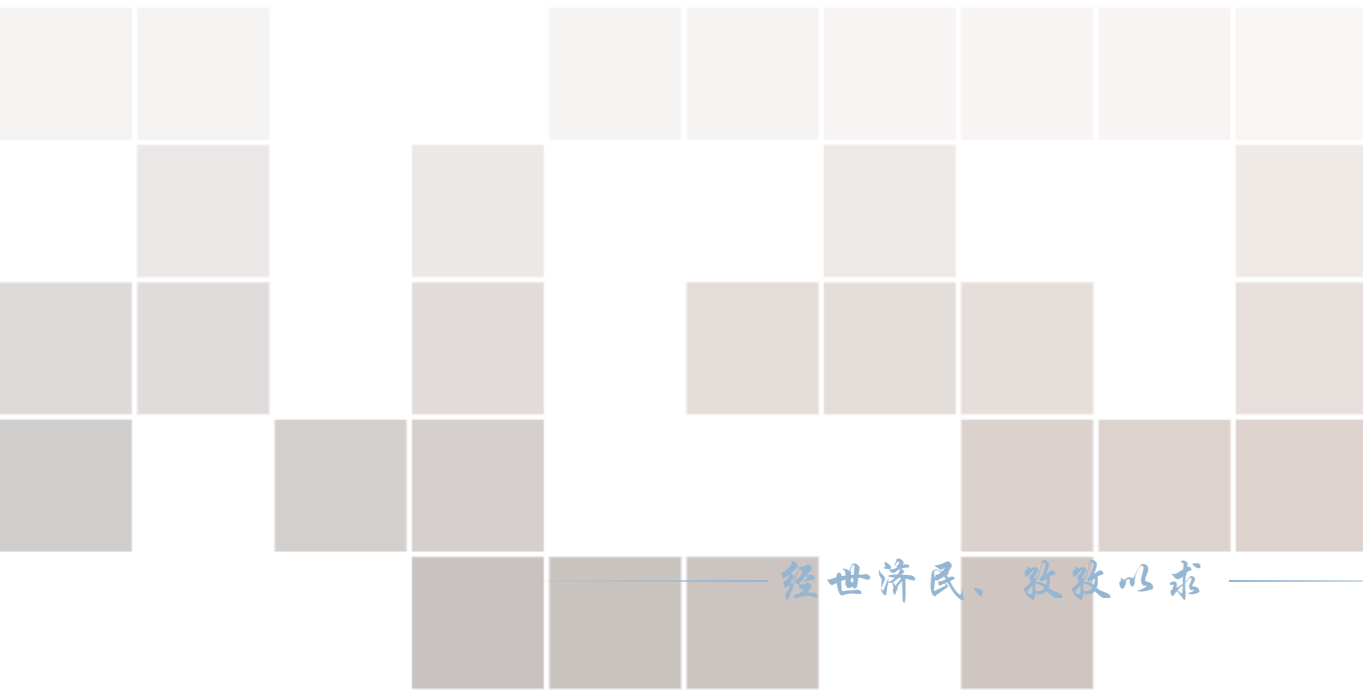
四、动态规划算法的例子

图像压缩

最大子段和

最优二叉检索树概念

最优二叉检索树算法



● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

● 小结

图像压缩

像素点灰度值：0~255，为8位二进制数

图像的**灰度值序列**： $\{p_1, p_2, \dots, p_n\}$, p_i 为第*i*个像素点灰度值

图像存储：每个像素的灰度值占8位，总计空间位 $8n$



有没有更好的存储方法呢？



● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

● 小结

图像变位压缩的概念

变位压缩存储：将 $\{p_1, p_2, \dots, p_n\}$ 分为 m 段

$$S_1, S_2, \dots, S_m$$



同一段的像素占用位数相同

第 t 段有 $l[t]$ 个像素(技术限制最大为256位), 每个占用 $b[t]$ 位

段头：记录 $l[t]$ (8位)和 $b[t]$ (3位)需要11位

总位数为：

$$b[1] * l[1] + b[2] * l[2] + \dots + b[m] * l[m] + 11m$$

● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

● 小结

图像压缩问题

约束条件：第 t 段像素个数 $l[t] \leq 256$

第 t 段占用空间： $b[t] \times l[t] + 11$

$$b[i] = \lceil \log(\max_{p_k \in S_i} p_k + 1) \rceil \leq 8$$

一段像素中灰度值最大项所需位数

问题：给定像素序列 $\{p_1, p_2, \dots, p_n\}$ ，确定最优分段，即

$$\min_T \left\{ \sum_{i=1}^j (b[i] \times l[i] + 11) \right\}$$

$T = \{S_1, S_2, \dots, S_j\}$ 为分段



● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

● 小结

实例

灰度值序列

$$P = \{10, 12, 15, 255, 1, 2, 1, 1, 2, 2, 1, 1\}$$

分法 1: $S_1 = \langle 10, 12, 15 \rangle$, $S_2 = \langle 255 \rangle$,
 $S_3 = \langle 1, 2, 1, 1, 2, 2, 1, 1 \rangle$

分法 2: $S_1 = \langle 10, 12, 15, 255, 1, 2, 1, 1, 2, 2, 1, 1 \rangle$

分法 3: 分成12组, 每组一个数

存储空间

分法 1: $11 \times 3 + 4 \times 3 + 8 \times 1 + 2 \times 8 = 69$

分法 2: $11 \times 1 + 8 \times 12 = 107$

分法 3: $11 \times 12 + 4 \times 3 + 8 \times 1 + 1 \times 5 + 2 \times 3 = 163$

● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

● 小结

子问题界定与计算顺序

子问题前边界为1, 后边界为 i

对应像素序列为 $\langle p_1, p_2, \dots, p_i \rangle$

优化函数值 $S[i]$ 为最优分段存储位数
计算顺序

$i=1$ 像素



$i=2$ 像素



...

$i=n$ 像素





● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

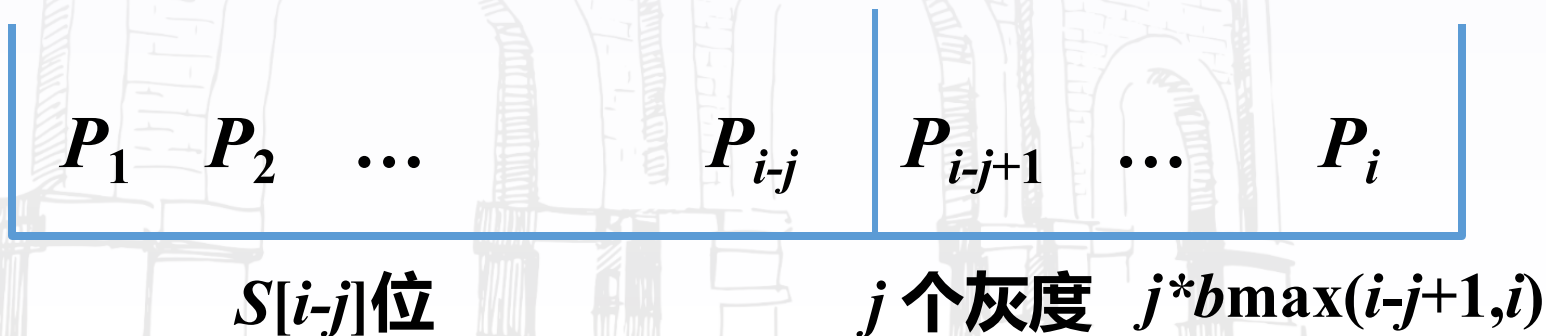
● 小结

算法设计

递推方程：设 $S[i]$ 是像素序列 $\langle p_1, p_2, \dots, p_i \rangle$ 的最优分段所需存储位数，分成 m 个段后， S_m 是最后分段

$$S[i] = \min_{1 \leq j \leq \min\{i, 256\}} \{S[i-j] + j \times b[i-j+1, i]\} + 11$$

$$b[i-j+1, i] = \lceil \log(\max_{p_k \in S_m} p_k + 1) \rceil \leq 8$$



● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

● 小结

算法 Compress (P, n)

//计算最小位数 $S[n]$

1. $Lmax \leftarrow 256$; $header \leftarrow 11$; $S[0] \leftarrow 0$ //最大段长 $Lmax$, 头 $header$

2. for $i \leftarrow 1$ to n do

子问题后
边界 i

3. $b[i] \leftarrow \text{length}(P[i])$ // $b[i]$ 是第 i 个灰度 $P[i]$ 的二进制位数

4. $bmax \leftarrow b[i]$ //3-6行分法最后一段只有 $P[i]$ 自己

5. $S[i] \leftarrow S[i-1] + bmax$

6. $l[i] \leftarrow 1$

最后段长
 j

7. for $j \leftarrow 2$ to $\min\{i, Lmax\}$ do //最后段含 j 个像素

8. if $bmax < b[i-j+1]$ //统一段内表示像素的二进制位数

9. then $bmax \leftarrow b[i-j+1]$

10. if $S[i] > S[i-j] + j * bmax$

11. then $S[i] \leftarrow S[i-j] + j * bmax$

找到更好
分段

12. $l[i] \leftarrow j$

13. $S[i] \leftarrow S[i] + header$



● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例分析

● 追踪解

● 小结

实例

$P = \langle 10, 12, 15, 255, 1, 2 \rangle$.

$S[1]=15, S[2]=19, S[3]=23, S[4]=42, S[5]=50,$

$l[1]=1, l[2]=2, l[3]=3, l[4]=1, l[5]=2$

10	12	15	255	1	2
----	----	----	-----	---	---

$S[5]=50$ $1 \times 2 + 11$ 63

10	12	15	255	1	2
----	----	----	-----	---	---

$S[4]=42$ $2 \times 2 + 11$ 57

10	12	15	255	1	2
----	----	----	-----	---	---

$S[3]=23$ $3 \times 8 + 11$ 58

10	12	15	255	1	2
----	----	----	-----	---	---

$S[2]=19$ $4 \times 8 + 11$ 62

10	12	15	255	1	2
----	----	----	-----	---	---

$S[1]=15$ $5 \times 8 + 11$ 66

10	12	15	255	1	2
----	----	----	-----	---	---

$6 \times 8 + 11$ 59



● 图像压缩

● 问题定义

● 变位压缩

● 问题定义

● 实例分析

● 子问题界定

● 算法设计

● 伪码

● 实例

● 追踪解

● 小结

追踪解

算法 $\text{Traceback}(n, l)$

输入：数组 l

输出：数组 C // $C[j]$ 是从后向前追踪的第 j 段的长度

1. $j \leftarrow 1$ // j 为正在追踪的段数

2. while $n \neq 0$ do

3. $C[j] \leftarrow l[n]$ 第 j 段长度

4. $n \leftarrow n - l[n]$

5. $j \leftarrow j + 1$

时间复杂度： $O(n)$



小结

图像变位存储问题的建模

- 子问题边界的界定
- 递推方程及初值
- 伪码
- 标记函数与解的追踪
- 时间复杂度



● 最大子段和

● 问题定义

● 算法对比

● 蛮力算法伪码

● 分治算法

● 分治算法伪码

● 分治复杂度

● 动态规划

● 递推方程

● 伪码

● 小结

最大子段和

问题： 给定n个数(可以为负数)的序列

$$\langle a_1, a_2, \dots, a_n \rangle$$

$$\text{求 } \max \{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \}$$

实例： $\langle -2, 11, -4, 13, -5, -2 \rangle$

解： 最大子段和 $a_2 + a_3 + a_4 = 20$



● 最大子段和

● 问题定义

● 算法对比

● 蛮力算法伪码

● 分治算法

● 分治算法伪码

● 分治复杂度

● 动态规划

● 递推方程

● 伪码

● 小结

算法对比

算法1---顺序求和+比较

对所有的 (i,j) 对, 顺序求和 $a_i + \dots + a_j$ 并比较出最大的和

算法2---分治策略

将数组分成左右两半, 分别计算左边的最大和、右边的最大和、跨边界的最大和, 然后比较其中最大者

算法3---动态规划



● 最大子段和

● 问题定义

● 算法对比

● 蛮力算法伪码

● 分治算法

● 分治算法伪码

● 分治复杂度

● 动态规划

● 递推方程

● 伪码

● 小结

算法1

Enumerate

输入：数组 $A[1..n]$

输出： $sum, first, last$

1. $sum \leftarrow 0$

和的边界
 $i-j$

2. for $i \leftarrow 1$ to n do // i 为当前和的首位置

3. for $j \leftarrow i$ to n do // j 为当前和的末位置

4. $thissum \leftarrow 0$ // $thissum$ 为 $A[i]$ 到 $A[j]$ 之和

5. for $k \leftarrow i$ to j do

6. $thissum \leftarrow thissum + A[k]$

找到更大的
和

7. if $thissum > sum$

8. then $sum \leftarrow thissum$

9. $first \leftarrow i$ // 记录最大和的首位置

10. $last \leftarrow j$ // 记录最大和的末位置

● 最大子段和

● 问题定义

● 算法对比

● 蛮力算法伪码

● 分治算法

● 分治算法伪码

● 分治复杂度

● 动态规划

● 递推方程

● 伪码

● 小结

算法2 分治策略

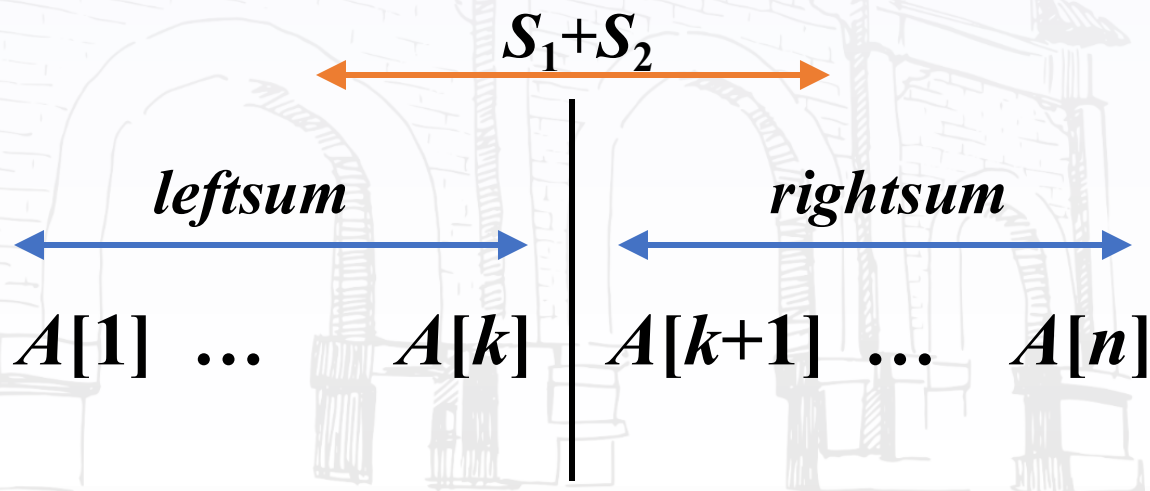
将序列分成左右两半，中间分点 $center$

递归计算左段最大子段和 $leftsum$

递归计算右段最大子段和 $rightsum$

$a_{center} \rightarrow a_1$ 的最大和 S_1 , $a_{center+1} \rightarrow a_n$ 的最大和 S_2

$\max \{ leftsum, rightsum, S_1+S_2 \}$





● 最大子段和

伪码

算法 $\text{MaxSubSum}(A, \text{left}, \text{right})$

输入：数组 A , left , right 分别是 A 的左、右边界

输出： A 的最大子段和 sum 及其子段的前后边界

1. if $|A| = 1$ then 输出元素值（当值为负时输出0）
2. $\text{center} \leftarrow \lfloor (\text{left} + \text{right}) / 2 \rfloor$
3. $\text{leftsum} \leftarrow \text{MaxSubSum}(A, \text{left}, \text{center})$ //子问题 A_1
4. $\text{rightsum} \leftarrow \text{MaxSubSum}(A, \text{center} + 1, \text{right})$ //子问题 A_2
5. $S_1 \leftarrow A_1[\text{left}, \text{center}]$ //从 center 向左的最大和
6. $S_2 \leftarrow A_2[\text{center} + 1, \text{right}]$ //从 $\text{center} + 1$ 向右的最大和
7. $\text{sum} \leftarrow S_1 + S_2$
8. if $\text{leftsum} > \text{sum}$ then $\text{sum} \leftarrow \text{leftsum}$
9. if $\text{rightsum} > \text{sum}$ then $\text{sum} \leftarrow \text{rightsum}$



● 最大子段和

● 问题定义

● 算法对比

● 蛮力算法伪码

● 分治算法

● 分治算法伪码

● 分治复杂度

● 动态规划

● 递推方程

● 伪码

● 小结

时间复杂度

计算从 $k=center$ 开始到 a_1 方向的最大和，每次加1个元素，得到：

$$A[k], A[k] + A[k-1], A[k] + A[k-1] + A[k-2], \dots, A[k] + \dots + A[1]$$

比较上述的最大和，时间为 $O(n)$ ，右半边也是 $O(n)$

$$T(n) = 2T(n/2) + O(n)$$

$$T(c) = O(1)$$

$$T(n) = O(n \log n)$$

● 最大子段和

● 问题定义

● 算法对比

● 蛮力算法伪码

● 分治算法

● 分治算法伪码

● 分治复杂度

● 动态规划

● 递推方程

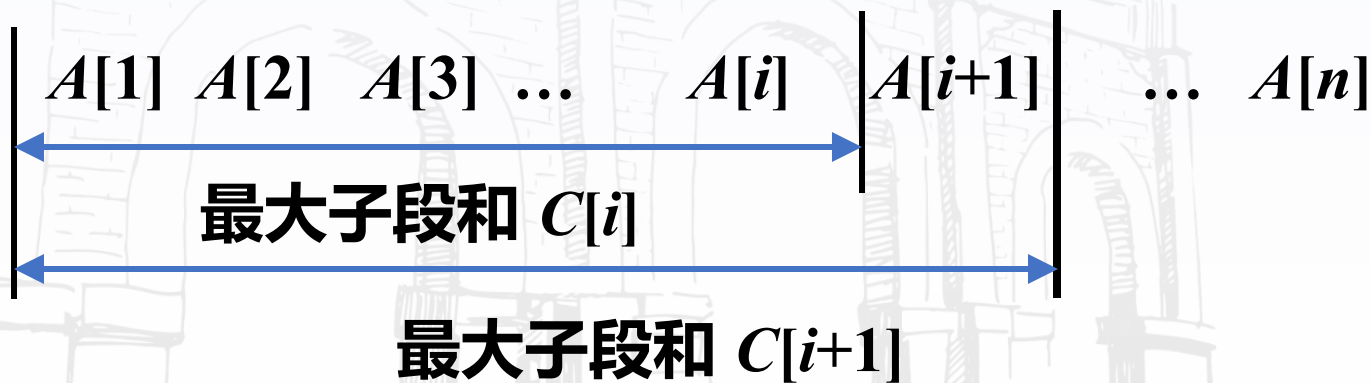
● 伪码

● 小结

算法3：动态规划

子问题界定：前边界为1，后边界为 i ， $C[i]$ 是 $A[1..i]$ 中**必须包含元素 $A[i]$** 的向前连续延伸的最大子段和

$$C[i] = \max_{1 \leq k \leq i} \left\{ \sum_{j=k}^i A[j] \right\}$$





最大子段和

- 问题定义
- 算法对比
- 蛮力算法伪码
- 分治算法
- 分治算法伪码
- 分治复杂度
- 动态规划
- 递推方程
- 伪码
- 小结

优化函数的递推方程

递推方程:

$$C[i+1] = \max\{C[i] + A[i+1], A[i+1]\}$$

$$i = 1, 2, \dots, n$$

$$C[1] = A[1] \quad \text{若 } A[1] > 0,$$

$$C[1] = 0 \quad \text{否则}$$

解: $\text{OPT}(A) = \max_{1 \leq i \leq n} \{C[i]\}$



最大子段和

问题定义

算法对比

蛮力算法伪码

分治算法

分治算法伪码

分治复杂度

动态规划

递推方程

伪码

小结

伪码

MaxSum (A, n)

输入：数组 A

输出：最大子段和 sum , 子段的最后位置 c

1. $sum \leftarrow 0$
2. $b \leftarrow 0$ // b 是前一个最大子段和
3. for $i \leftarrow 1$ to n do
4. if $b > 0$
5. then $b \leftarrow b + A[i]$
6. else $b \leftarrow A[i]$
7. if $b > sum$
8. then $sum \leftarrow b$
9. $c \leftarrow i$ //记录最大和的末项标号
10. return sum, c

子问题后
边界

找到更大的
和



小结

- 几个算法：蛮力，分治，动态规划
- 动态规划算法：

子问题界定

列优化函数的递推方程和边界条件 (不一定是原问题的优化函数)

自底向上计算，设计备忘录（表格）

如何根据动态规划的解找原问题的解

时间复杂度估计

● 最优二叉检索

● 问题定义

● 检索方法

● 数据存储分布

● 实例

● 检索平均时间1

● 检索平均时间2

● 平均比较次数

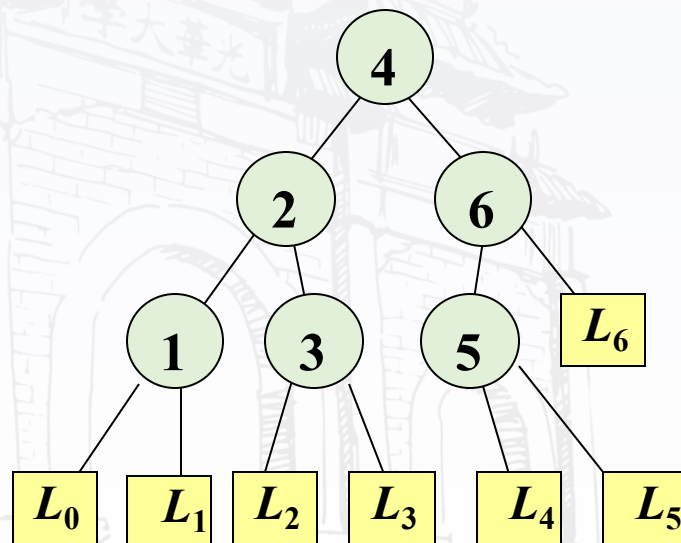
● 小结

最优二叉检索树

设集合 S 为排序的 n 个元素 $x_1 < x_2 < \dots < x_n$, 将这些元素存储在一棵二叉树的结点上, 以查找 x 是否在这些数中. 如果 x 不在, 确定 x 在那个空隙.

实例:

$S = \{ 1, 2, 3, 4, 5, 6 \}$



最优二叉检索

问题定义

检索方法

数据存储分布

实例

检索平均时间1

检索平均时间2

平均比较次数

小结

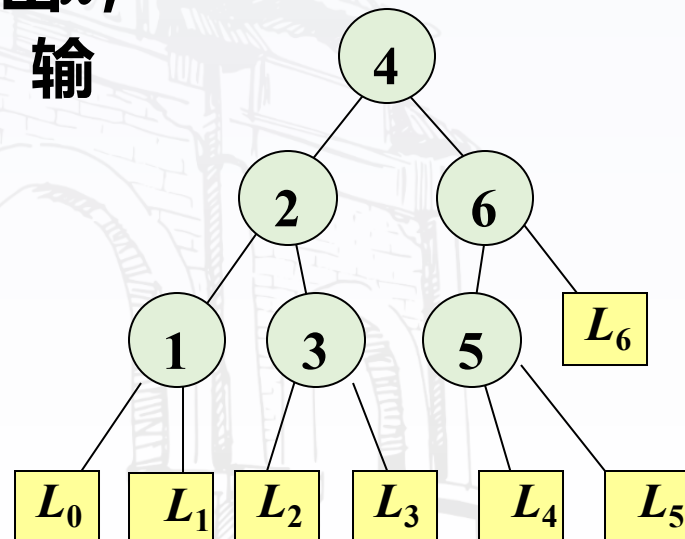
最优二叉检索方法

检索方法:

1. 初始, x 与根元素比较;
2. $x <$ 根元素, 递归进入左子树;
3. $x >$ 根元素, 递归进入右子树;
4. $x =$ 根元素, 算法停止, 输出 x ;
5. x 到达叶结点, 算法停止, 输出 x 不在数组中.

$$x = 3.5$$

1	2	3	4	5	6
---	---	---	---	---	---





● 最优二叉检索

● 问题定义

● 检索方法

● 数据存储分布

● 实例

● 检索平均时间1

● 检索平均时间2

● 平均比较次数

● 小结

数据元素存储概率分布

空隙: $(-\infty, x_1), (x_1, x_2), \dots, (x_{n-1}, x_n), (x_n, +\infty)$,
 $x_0 = -\infty, x_{n+1} = +\infty$

给定序列 $S = \langle x_1, x_2, \dots, x_n \rangle$,

x 在 x_i 的概率为 b_i , x 在 (x_i, x_{i+1}) 的概率为 a_i ,

S 的存取概率分布如下:

$$P = \langle a_0, b_1, a_1, b_2, a_2, \dots, b_n, a_n \rangle$$



● 最优二叉检索

● 问题定义

● 检索方法

● 数据存储分布

● 实例

● 检索平均时间1

● 检索平均时间2

● 平均比较次数

● 小结

实例

实例

$S=\{1,2,3,4,5,6\}$

$P=<0.04, \mathbf{0.1}, 0.01, \mathbf{0.2}, 0.05, \mathbf{0.2}, 0.02, \mathbf{0.1}, 0.02, \mathbf{0.1}, 0.07, \mathbf{0.05}, 0.04>$

1, 2, 3, 4, 5, 6检索的概率分别为:

$\mathbf{0.1, 0.2, 0.2, 0.1, 0.1, 0.05}$

各个空隙的检索概率分别为:

$0.04, 0.01, 0.05, 0.02, 0.02, 0.07, 0.04$

● 最优二叉检索

● 问题定义

● 检索方法

● 数据存储分布

● 实例

● 检索平均时间1

● 检索平均时间2

● 平均比较次数

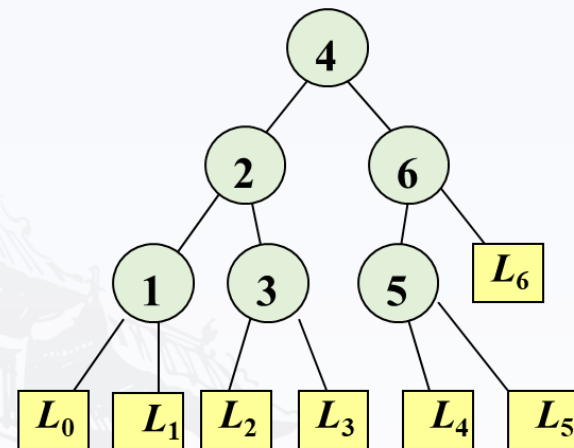
● 小结

检索数据的平均时间-1

实例

$$S=\{1,2,3,4,5,6\}$$

$$P=<0.04, \mathbf{0.1}, 0.01, \mathbf{0.2}, 0.05, \mathbf{0.2}, 0.02, \\ \mathbf{0.1}, 0.02, \mathbf{0.1}, 0.07, \mathbf{0.05}, 0.04>$$



$$\begin{aligned} m(T_1) &= [1*0.1 + 2*(0.2+0.05) + 3*(0.1+0.2+0.1)] \\ &\quad + [3*(0.04+0.01+0.05+0.02+0.02+0.07) + 2*0.04] \\ &= 1.8 + 0.71 = \mathbf{2.51} \end{aligned}$$

● 最优二叉检索

● 问题定义

● 检索方法

● 数据存储分布

● 实例

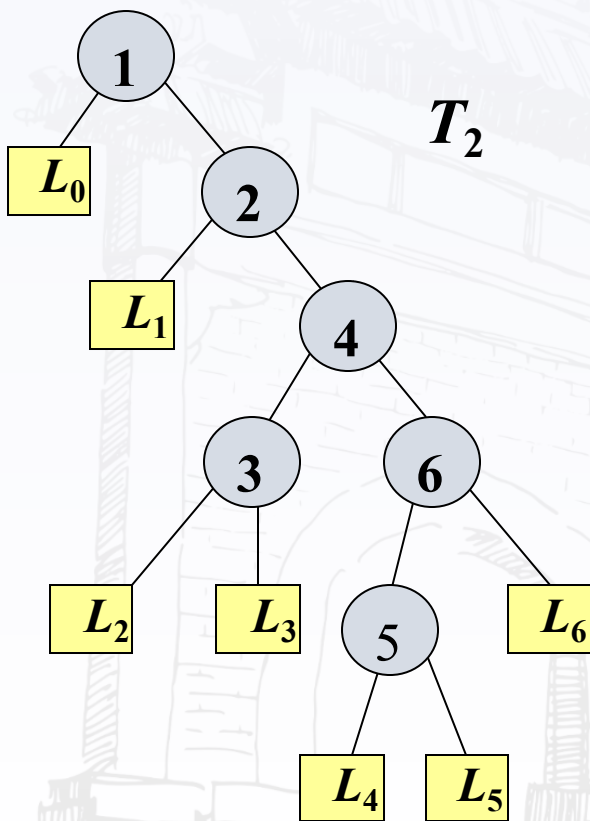
● 检索平均时间1

● 检索平均时间2

● 平均比较次数

● 小结

检索数据的平均时间-2



实例

$$S=\{1,2,3,4,5,6\}$$

$$P=<0.04,\mathbf{0.1},0.01,\mathbf{0.2},0.05,\mathbf{0.2},0.02,\mathbf{0.1},0.07,\mathbf{0.05},0.04>$$

$$\begin{aligned}
 m(T_2) &= \\
 &[1*0.1+2*0.2+3*0.1+4*(0.2+0.05)+5*0.1] \\
 &+ [1*0.04+2*0.01+4*(0.05+0.02+0.04)+ \\
 &5*(0.02+0.07)] \\
 &= 2.3+0.95=\mathbf{3.25}
 \end{aligned}$$

● 最优二叉检索

● 问题定义

● 检索方法

● 数据存储分布

● 实例

● 检索平均时间1

● 检索平均时间2

● 平均比较次数

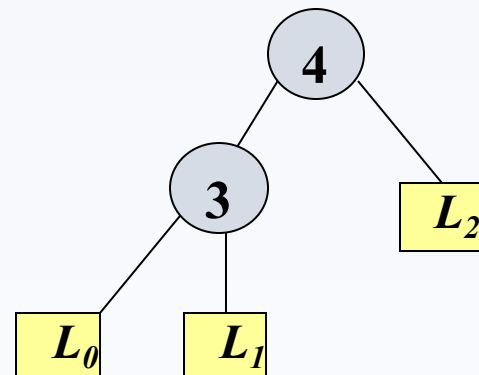
● 小结

平均比较次数计算

数据集 $S = \langle x_1, x_2, \dots, x_n \rangle$

存取概率分布

$P = \langle a_0, b_1, a_1, b_2, \dots, a_i, b_{i+1}, \dots, b_n, a_n \rangle$



结点 x_i 在 T 中的深度是 $d(x_i)$, $i = 1, 2, \dots, n$,
空隙 L_j 的深度为 $d(L_j)$, $j = 0, 1, \dots, n$,

平均比较次数为:

$$t = \sum_{i=1}^n b_i (1 + d(x_i)) + \sum_{j=0}^n a_j d(L_j)$$



小结

- 二叉检索树的构成
- 给定概率分布下，一颗二叉检索树的平均检索时间估计
- 什么是最优二叉检索树



● 二叉检索算法

● 涉及的问题

● 子问题划分

● 子问题规约

● 子问题概率和

● 递推方程

● 公式证明

● 递推方程

● 实例

● 复杂性估计

● 小结

最优二叉检索树的算法-关键问题

- ① 子问题边界界定
- ② 如何将该问题归结为更小的子问题
- ③ 优化函数的递推方程及初值
- ④ 计算顺序
- ⑤ 是否需要标记函数
- ⑥ 时间复杂度分析



● 二叉检索算法

● 涉及的问题

● 子问题划分

● 子问题规约

● 子问题概率和

● 递推方程

● 公式证明

● 递推方程

● 实例

● 复杂性估计

● 小结

子问题划分

子问题边界为 (i, j) *原始问题边界为 $(1, n)$

数据集: $S[i, j] = \langle x_i, x_i + 1, \dots, x_j \rangle$

存取概率分布:

$$P[i, j] = \langle a_{i-1}, b_i, a_i, b_{i+1}, \dots, b_j, a_j \rangle$$

输入实例: $S = \langle A, B, C, D, E \rangle$ *选取子问题BCD构造二叉树

$$P = \langle 0.04, \mathbf{0.1}, 0.02, \mathbf{0.3}, 0.02, \mathbf{0.1}, 0.05, \mathbf{0.2}, 0.06, \mathbf{0.1}, 0.01 \rangle$$

子问题: $S[2, 4] = \langle B, C, D \rangle$

$$P = \langle 0.02, \mathbf{0.3}, 0.02, \mathbf{0.1}, 0.05, \mathbf{0.2}, 0.06 \rangle$$

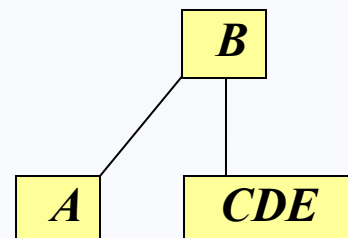
● 二叉检索算法

子问题规约

以 x_k 作为根归结为子问题:

$$S[i, k - 1], P[i, k - 1]$$

$$S[k + 1, j], P[k + 1, j]$$



$$S[1, 5] = \langle A, B, C, D, E \rangle$$

$$P[1, 5] = \langle 0.04, \mathbf{0.1}, 0.02, \mathbf{0.3}, 0.02, \mathbf{0.1}, 0.05, \mathbf{0.2}, 0.06, \mathbf{0.1}, 0.01 \rangle$$

$$S[1, 1] = \langle A \rangle$$

$$P[1, 1] = \langle 0.04, \mathbf{0.1}, 0.02 \rangle$$

$$S[3, 5] = \langle C, D, E \rangle$$

$$P[3, 5] = \langle 0.02, \mathbf{0.1}, 0.05, \mathbf{0.2}, 0.06, \mathbf{0.1}, 0.01 \rangle$$



● 二叉检索算法

● 涉及的问题

● 子问题划分

● 子问题规约

● 子问题概率和

● 递推方程

● 公式证明

● 递推方程

● 实例

● 复杂性估计

● 小结

子问题的概率之和

子问题界定 $S[i, j]$ 和 $P[i, j]$, 令

$$w[i, j] = \sum_{p=i-1}^j a_p + \sum_{q=i}^j b_q$$

是 $P[i, j]$ 中所有概率(数据与空隙)之和

实例: $S[2, 4] = \langle B, C, D \rangle$

$P[2, 4] = \langle 0.02, 0.3, 0.02, 0.1, 0.05, 0.2, 0.06 \rangle$

$W[2, 4]$

$= (0.3 + 0.1 + 0.2) + (0.02 + 0.02 + 0.05 + 0.06)$

$= 0.75$

二叉检索算法

- 涉及的问题
- 子问题划分
- 子问题规约
- 子问题概率和
- 递推方程
- 公式证明
- 递推方程
- 实例
- 复杂性估计
- 小结

优化函数的递推方程

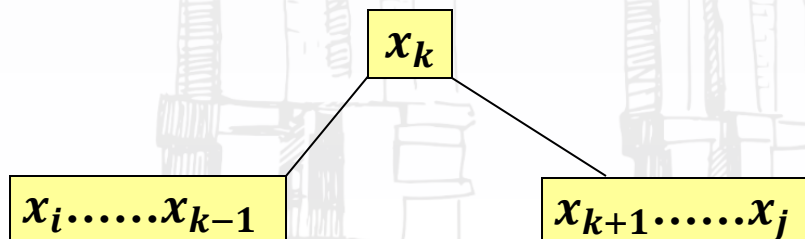
设 $m[i, j]$ 是相对于输入 $S[i, j]$ 和 $P[i, j]$ 的最优二叉搜索树的平均比较次数

递推方程:

$$m[i, j] = \min_{i \leq k \leq j} \{ \underline{m[i, k-1]} + \underline{m[k+1, j]} + w[i, j] \},$$

$$1 \leq i \leq j \leq n$$

$$m[i, i-1] = 0, \quad i = 1, 2, \dots, n$$



● 二叉检索算法

● 涉及的问题

● 子问题划分

● 子问题规约

● 子问题概率和

● 递推方程

● 公式证明

● 递推方程

● 实例

● 复杂性估计

● 小结

$m[i, j]_k$ 公式的证明

$m[i, j]_k$: 根为 x_k 时的二分检索树平均比较次数的最小值

$$\begin{aligned}
 & m[i, j]_k \\
 &= (m[i, k-1] + \underline{w[i, k-1]}) + (\underline{m[k+1, j] + w[k+1, j]}) + 1 \times b_k \\
 &= (m[i, k-1] + m[k+1, j]) + (w[i, k-1] + b_k + w[k+1, j]) \\
 &= (m[i, k-1] + m[k+1, j]) + (\underbrace{\sum_{p=i-1}^{k-1} a_p + \sum_{q=i}^{k-1} b_q}_{b_k}) + (\underbrace{\sum_{p=k}^j a_p + \sum_{q=k+1}^j b_q}_{b_k}) \\
 &= (m[i, k-1] + m[k+1, j]) + \sum_{p=i-1}^j a_p + \sum_{q=i}^j b_q \quad \text{化简} \\
 &= m[i, k-1] + m[k+1, j] + w[i, j]
 \end{aligned}$$

化简

二叉检索算法

递推方程

$$m[i, j] = \min_{i \leq k \leq j} \{m[i, k-1] + m[k+1, j] + w[i, j]\}$$

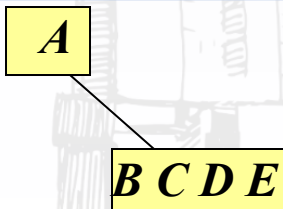
平均比较次数：在所有 k 的情况下 $m[i, j]_k$ 的最小值

$$m[i, j] = \min \{ m[i, j]_k \mid i \leq k \leq j \}$$

Note: 初值问题如何考虑?

初值 $m[i, i-1] = 0$ 对应于空的子问题,

例如 $S = \langle A, B, C, D, E \rangle$, 取 A 作根, $i = 1, k = 1$, 左边子问题为空树, 对应于: $S[1, 0], m[1, 0] = 0$ 的情况.



● 二叉检索算法

实例

$$m[i, j] = \min_{i \leq k \leq j} \{m[i, k-1] + m[k+1, j] + w[i, j]\} \quad 1 \leq i \leq j \leq n$$

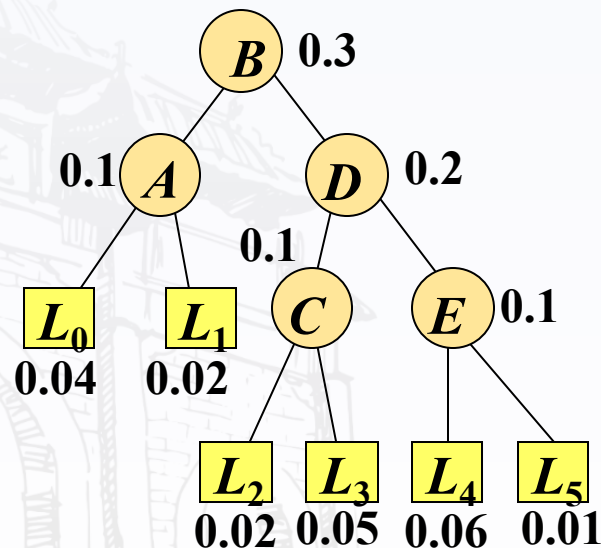
$$m[i, i-1] = 0 \quad i = 1, 2, \dots, n$$

以B为根

$$k = 2$$

$$m[1, 1] = 0.16$$

$$m[3, 5] = 0.88$$



$$m[1, 5] = 1 + \min_{k=2,3,4} \{m[1, k-1] + m[k+1, 5]\}$$

$$= 1 + \{m[1, 1] + m[3, 5]\} = 1 + \{0.16 + 0.88\} = 2.04$$



二叉检索算法

- 涉及的问题
- 子问题划分
- 子问题规约
- 子问题概率和
- 递推方程
- 公式证明
- 递推方程
- 实例
- 复杂性估计
- 小结

计算复杂性估计

$$m[i, j] = \min_{i \leq k \leq j} \{m[i, k-1] + m[k+1, j] + w[i, j]\} \quad 1 \leq i \leq j \leq n$$

$$m[i, i-1] = 0 \quad i = 1, 2, \dots, n$$

i, j 的所有组合 $O(n^2)$ 种

每种要对不同的 k 进行计算, $k = O(n)$

时间复杂性:

$$T(n) = O(n^3)$$

空间复杂性:

$$S(n) = O(n^2)$$



小结

- 划分子问题，以数据节点作为树根
- 定义优化函数，列出递推方程与边界条件
- 自底向上计算，设计备忘录（表格）
- 设立标记函数记录构成最优二叉搜索树或子树时根的位置.
- 时间复杂度估计



作业：第三章课后习题3.1

截止时间：4月27日提交至QQ作业